

UNIVERSITÀ
DI TORINO

DIPARTIMENTO DI
INFORMATICA

CORSO DI LAUREA
TRIENNALE

A.A. 2026/27

Canale B

Programmazione I

Teoria

Elvio Amparore

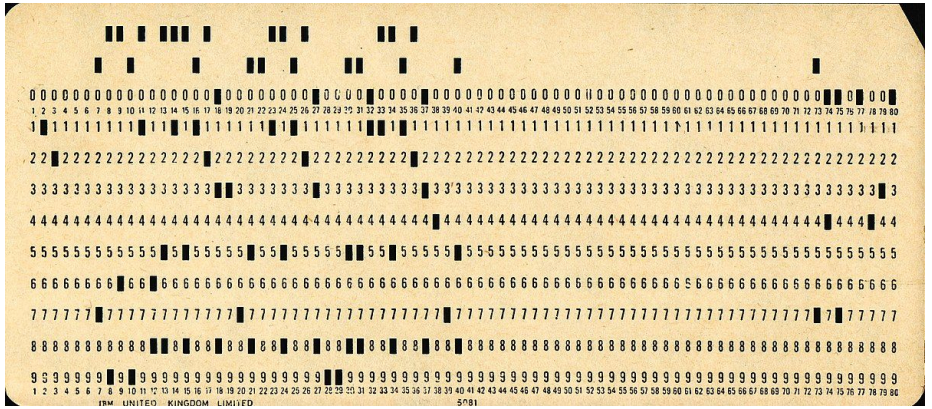
Dal linguaggio macchina al C

Dai bit e registri alla programmazione strutturata di alto livello e portabile

Dal codice macchina al linguaggio assembly

- I primi computer erano programmati direttamente in linguaggio macchina manipolando i bit dei registri tramite interruttori o schede perforate
- Simile all'esecuzione step-by-step utilizzata oggi per il debug dell'hardware in fase di progetto

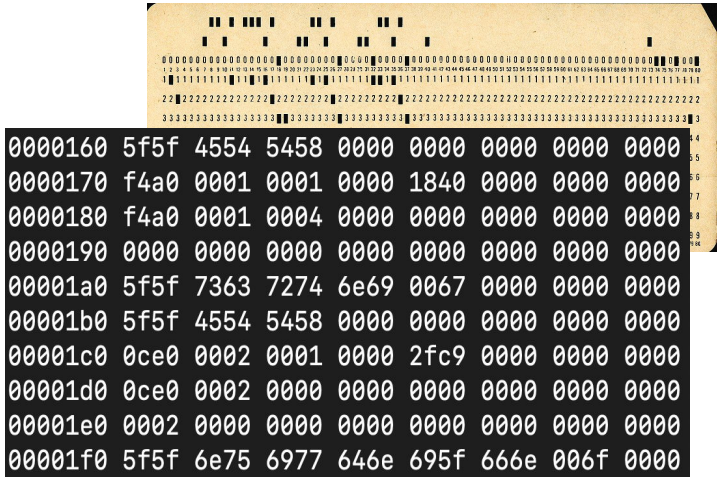




Vocabolario: linguaggio macchina e assembly

Il **linguaggio macchina** è il linguaggio direttamente eseguibile dal processore:

- è costituito da codici numerici, rappresentati come sequenze di bit, che identificano istruzioni e operandi;
- ogni architettura definisce il proprio insieme di istruzioni macchina (*instruction set*);
- dipende quindi dal processore e non è portabile tra architetture diverse.



```
0000160 5f5f 4554 5458 0000 0000 0000 0000 0000
0000170 f4a0 0001 0001 0000 1840 0000 0000 0000
0000180 f4a0 0001 0004 0000 0000 0000 0000 0000
0000190 0000 0000 0000 0000 0000 0000 0000 0000
00001a0 5f5f 7363 7274 6e69 0067 0000 0000 0000
00001b0 5f5f 4554 5458 0000 0000 0000 0000 0000
00001c0 0ce0 0002 0001 0000 2fc9 0000 0000 0000
00001d0 0ce0 0002 0000 0000 0000 0000 0000 0000
00001e0 0002 0000 0000 0000 0000 0000 0000 0000
00001f0 5f5f 6e75 6977 646e 695f 666e 006f 0000
```

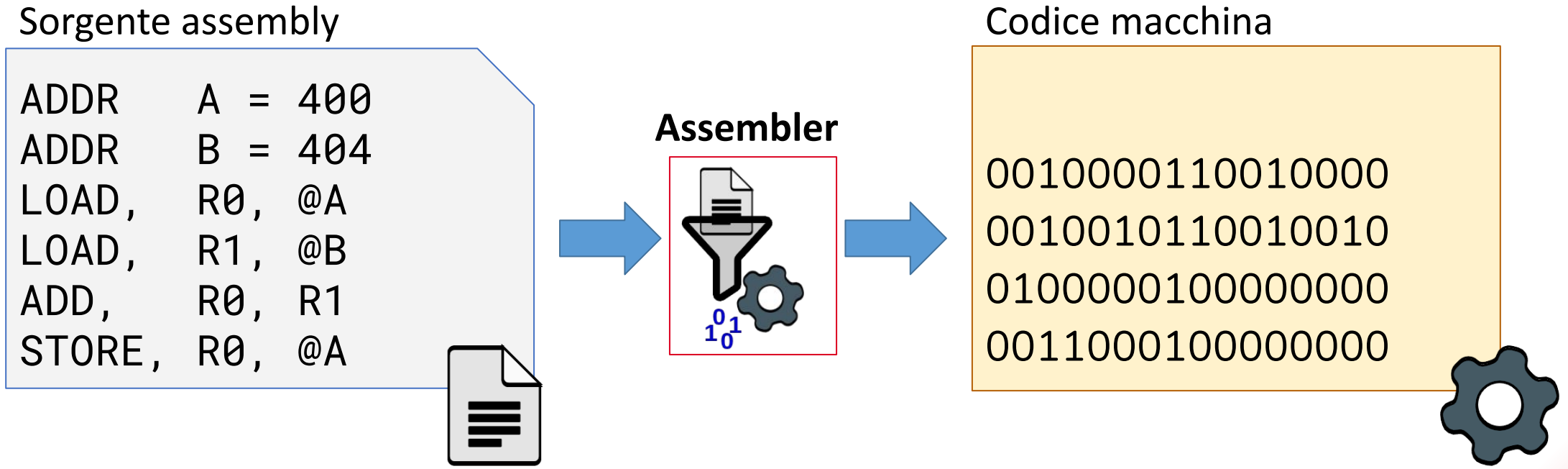
```
000000010001effc        ldr     w8, [sp, #0x14]
000000010001f000        add     w8, w8, #0x1
000000010001f004        str     w8, [sp, #0x14]
000000010001f008        b       0x10001eee8
000000010001f00c        ldr     w8, [sp, #0xc4]
000000010001f010        str     w8, [sp, #0x8]
000000010001f014        ldur    x9, [x29, #-0x18]
000000010001f018        adrp    x8, 5 ; 0x100024000
000000010001f01c        ldr     x8, [x8, #0x48] ; l
000000010001f020        ldr     x8, [x8]
```

Il **linguaggio assembly** è una rappresentazione testuale e simbolica del linguaggio macchina:

- usa mnemonici come `mov`, `add`, `ldr` al posto dei codici numerici delle istruzioni;
- viene tradotto in linguaggio macchina da un programma chiamato *assembler*;
- è più leggibile per il programmatore, ma rimane strettamente legato all'architettura hardware.

Dal codice macchina al linguaggio assembly

- Un software chiamato ***assembler*** ha il compito di tradurre (*compilare)* ogni istruzione di assembly in istruzione macchina



Esempio: addizionare tra interi

Vediamo un esempio con il linguaggio assembly.
Sommiamo due numeri interi in memoria, rispettivamente agli indirizzi A=400 e B=404, e mettiamo il risultato nella cella di memoria nell'indirizzo A

Il programma in linguaggio assembly (tipo RISC come MIPS32) avrà questa forma:

1. Definisce gli indirizzi A e B
2. Carica i numeri interi agli indirizzi A e B nei registri R0 ed R1 della CPU con due istruzioni **LOAD**
3. Esegue l'addizione tra i registri R0 ed R1 con una istruzione **ADD**
4. Copia il risultato dal registro R0 all'indirizzo A in memoria con una istruzione **STORE**

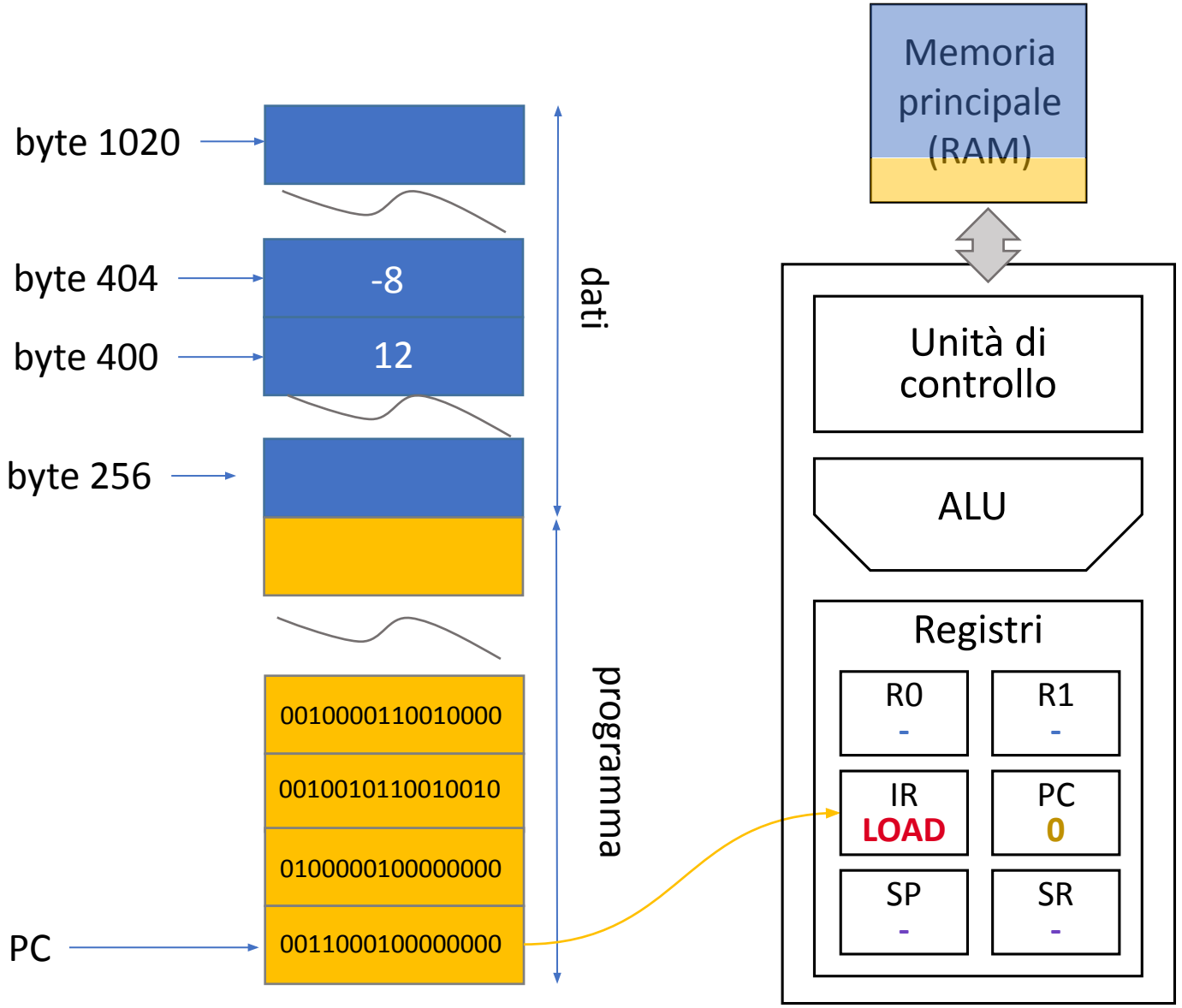
Esempio: addizione tra interi

Operazione	*Sorgente assembly*	*Linguaggio macchina (bits)*
Definisce i due indirizzi A e B	ADDR A = 400 ADDR B = 404	
Carica numero all'indirizzo A in R0	LOAD, R0, @A	001000000...0110010000
Carica numero all'indirizzo B in R1	LOAD, R1, @B	001000010...0110010010
Addiziona R0 ed R1, risultato in R0	ADD, R0, R1	010000000...0100000000
Copia risultato da R0 ad indirizzo A	STORE, R0, @A	001100000...0100000000

Esempio: addizione tra interi

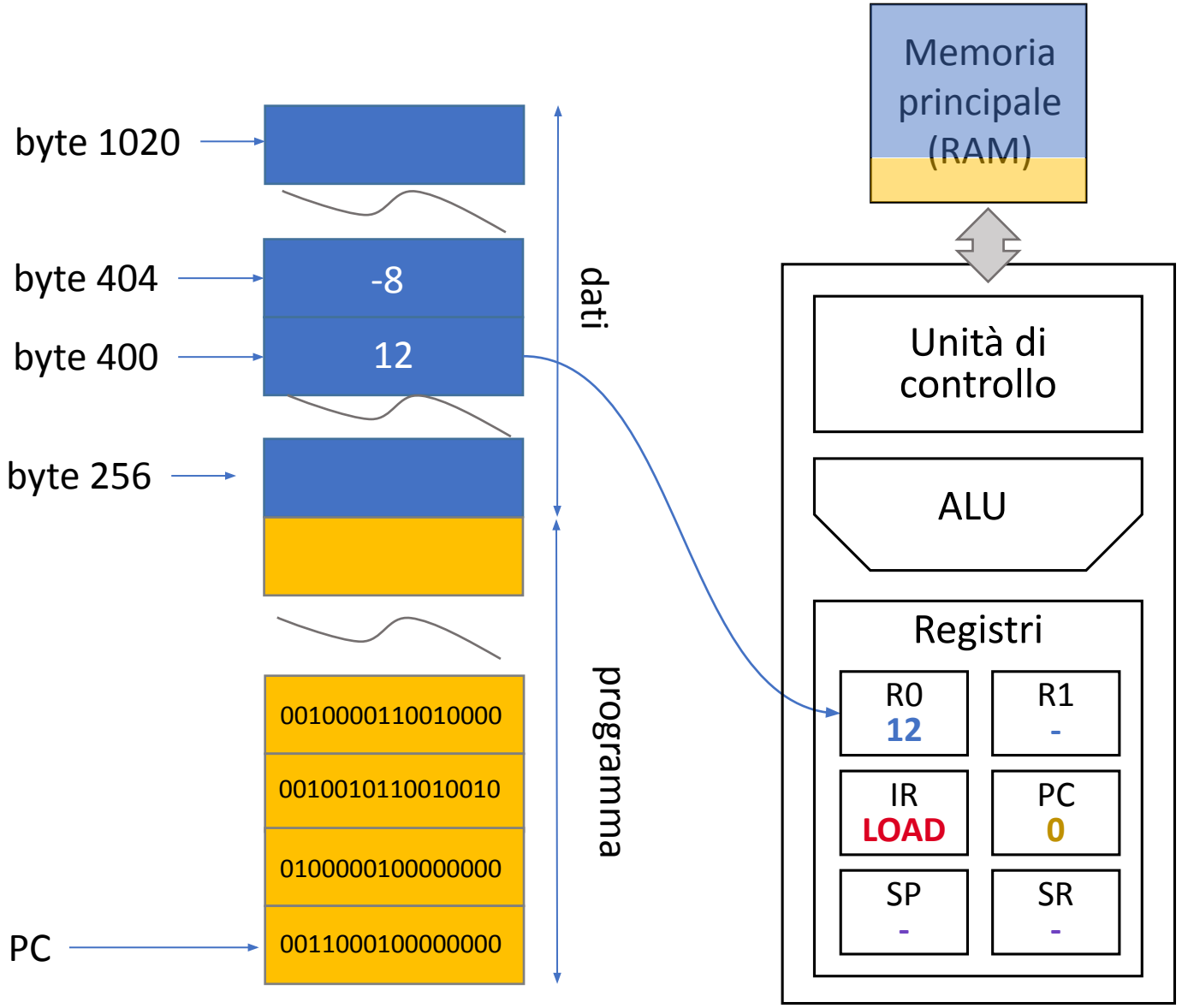
1. ▶ LOAD, R0, @A
2. LOAD, R1, @B
3. ADD, R0, R1
4. STORE, R0, @A

carica la prossima istruzione leggendo il program counter (PC) nell'instruction register (IR), per poterla eseguire.



Esempio: addizione tra interi

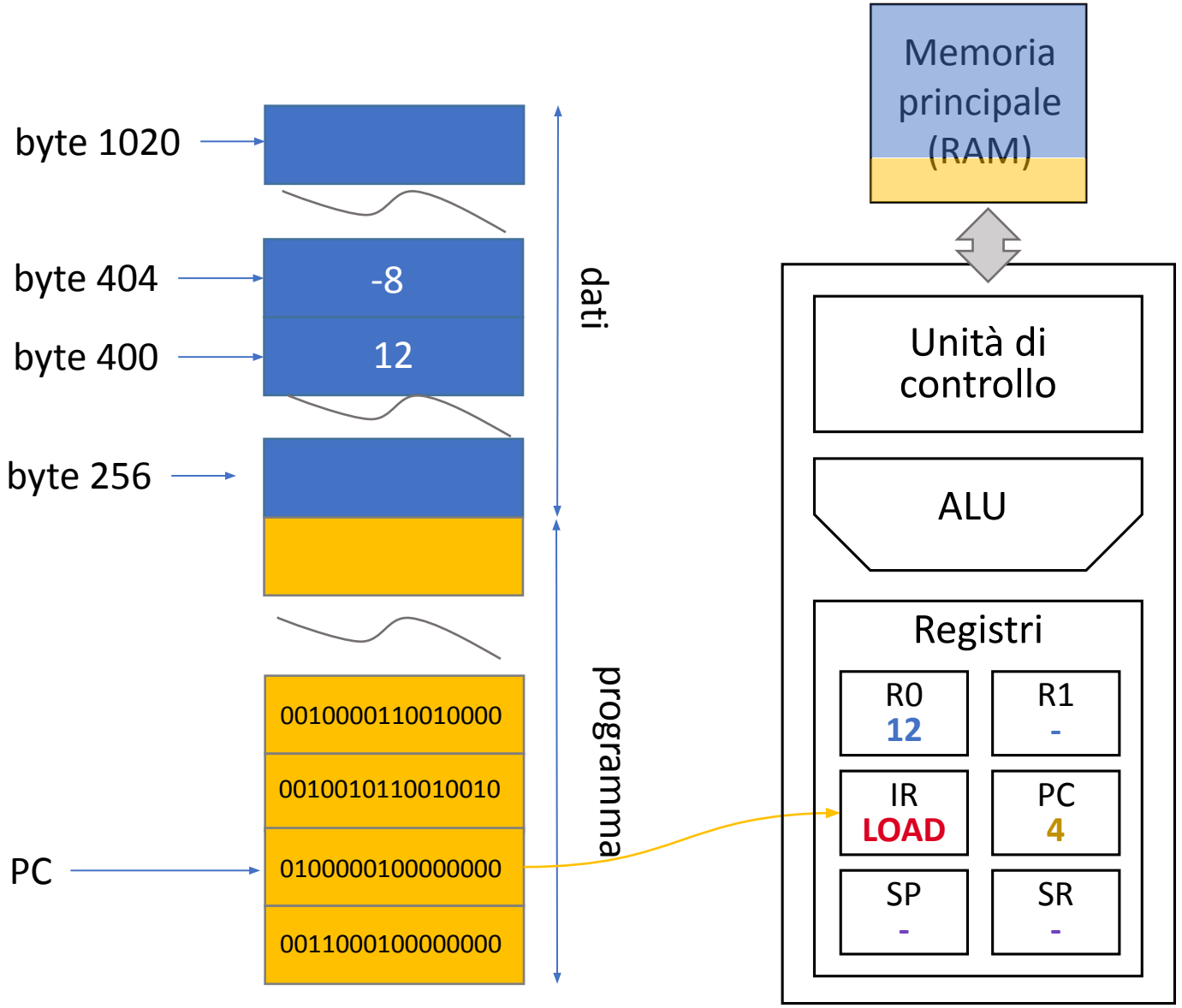
1. ▶ LOAD, R0, @A
2. LOAD, R1, @B
3. ADD, R0, R1
4. STORE, R0, @A



decodifica IR ed esegue l'istruzione elementare.

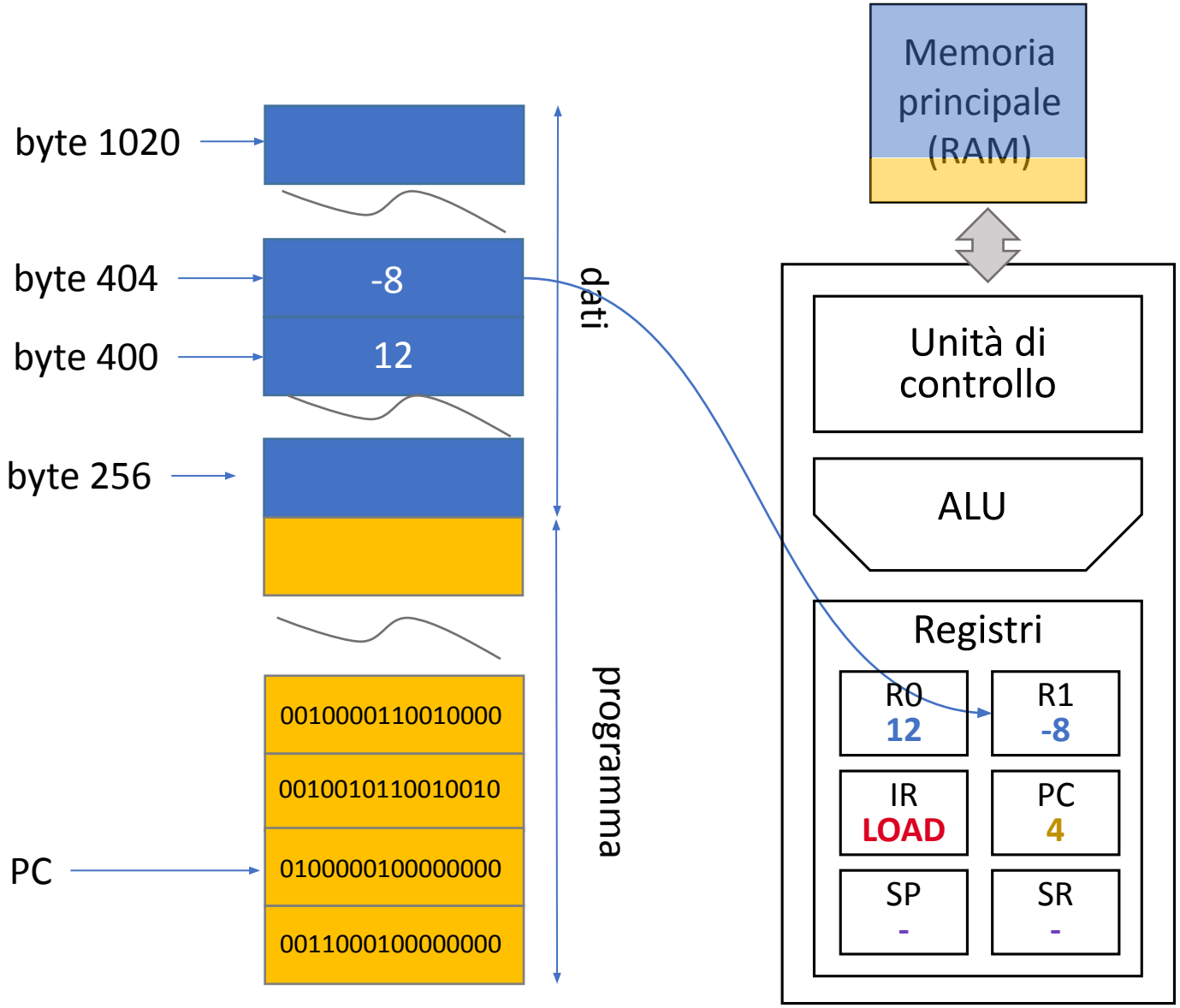
Esempio: addizione tra interi

1. LOAD, R0, @A
2. ▶ LOAD, R1, @B
3. ADD, R0, R1
4. STORE, R0, @A



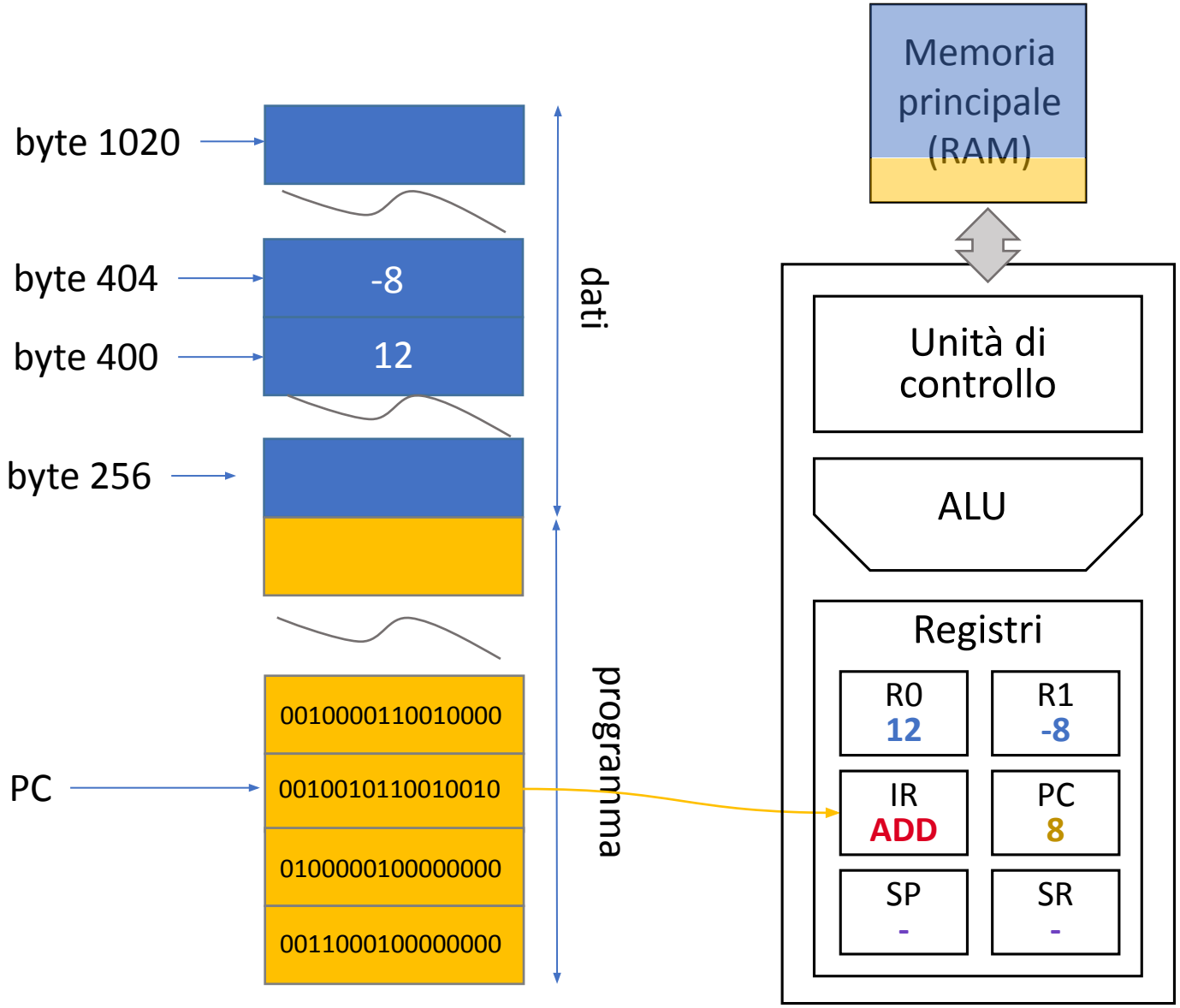
Esempio: addizione tra interi

1. LOAD, R0, @A
2. ▶ LOAD, R1, @B
3. ADD, R0, R1
4. STORE, R0, @A



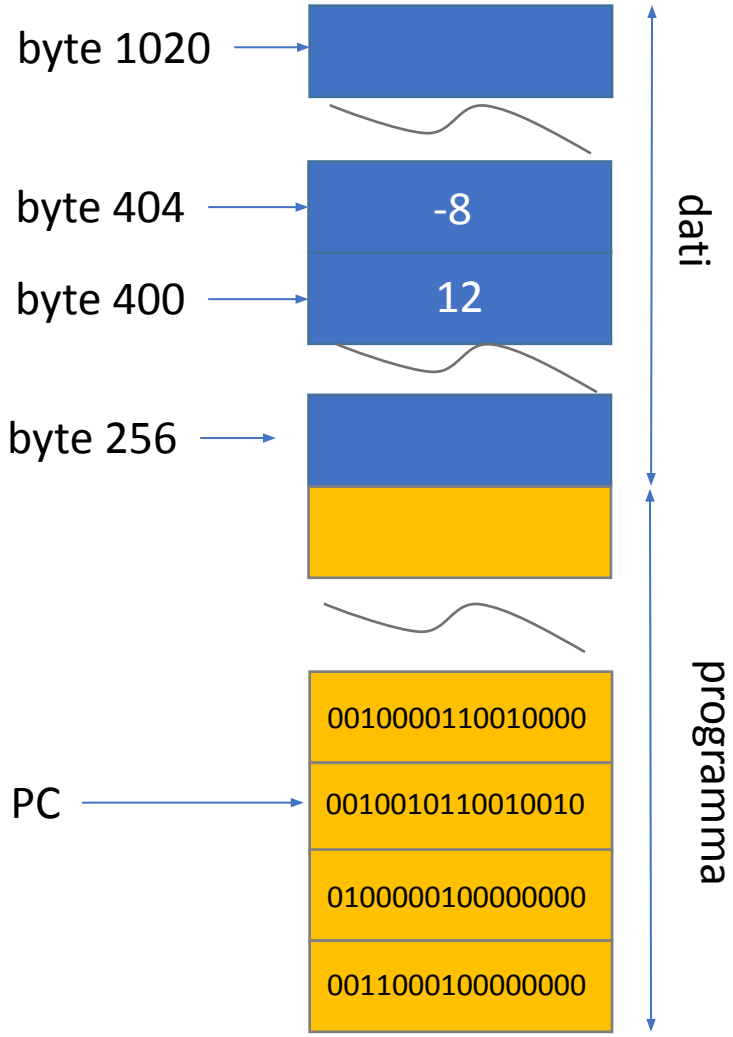
Esempio: addizione tra interi

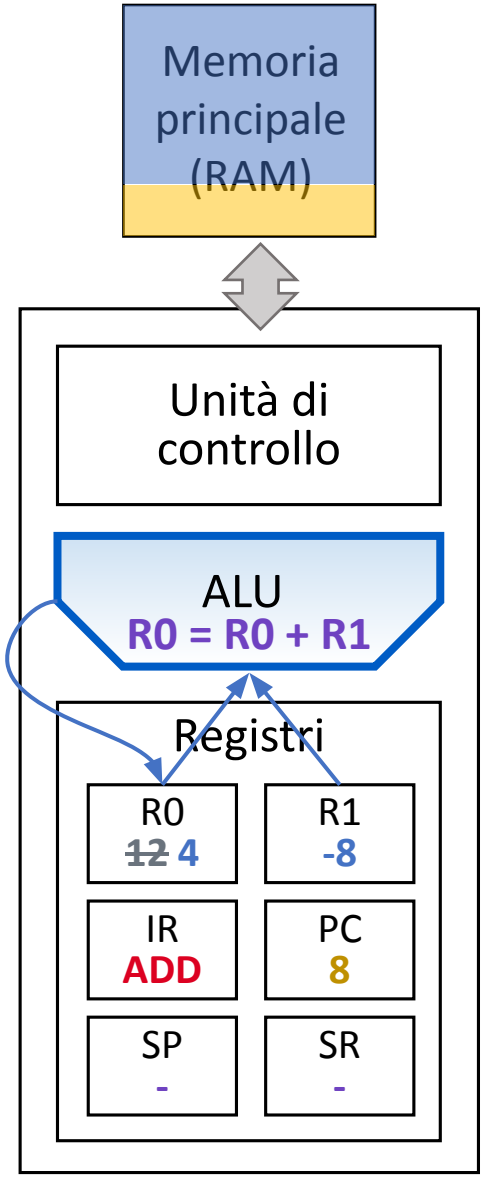
1. LOAD, R0, @A
2. LOAD, R1, @B
3. ▶ ADD, R0, R1
4. STORE, R0, @A



Esempio: addizione tra interi

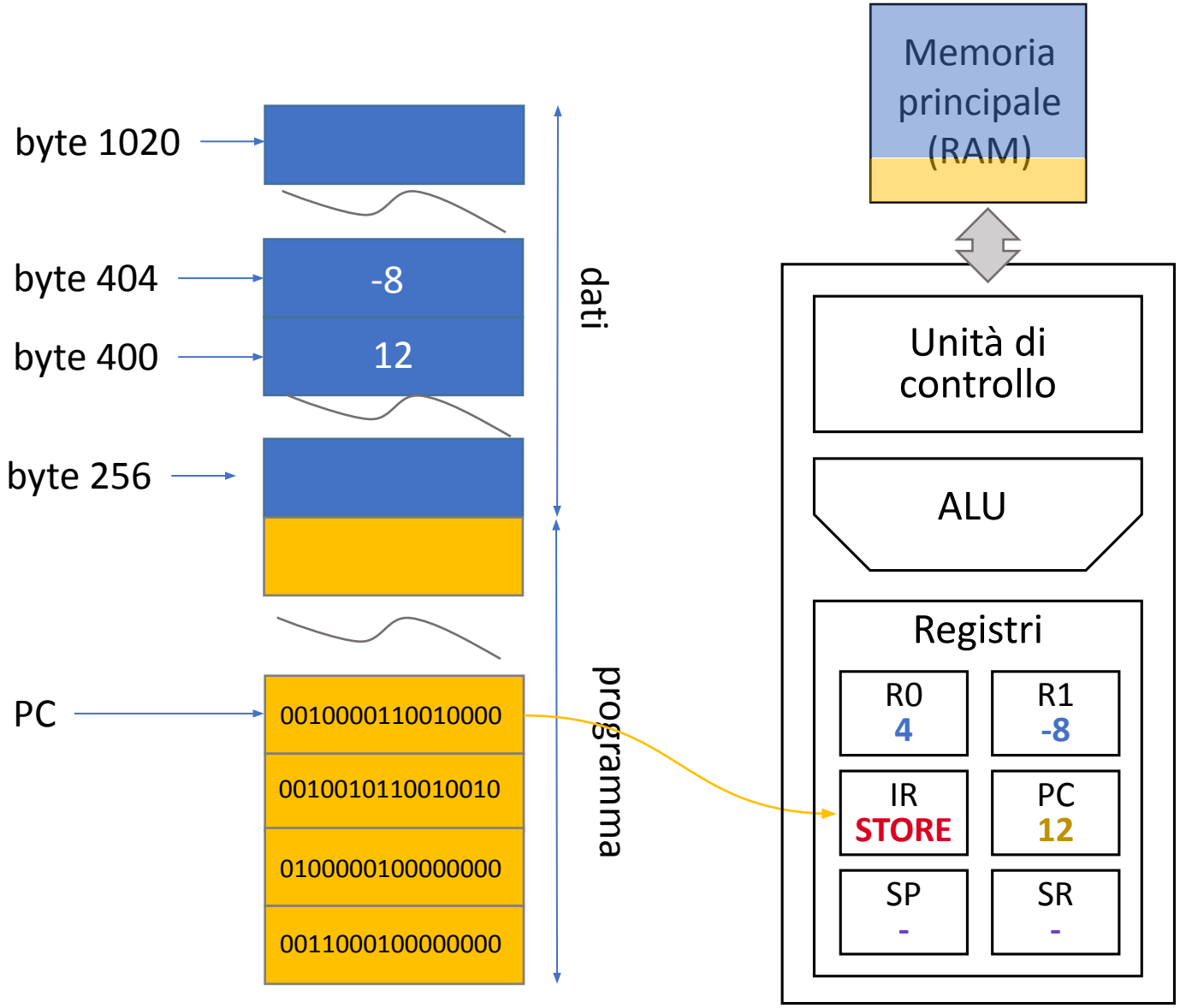
1. LOAD, R0, @A
2. LOAD, R1, @B
3. ▶ ADD, R0, R1
4. STORE, R0, @A





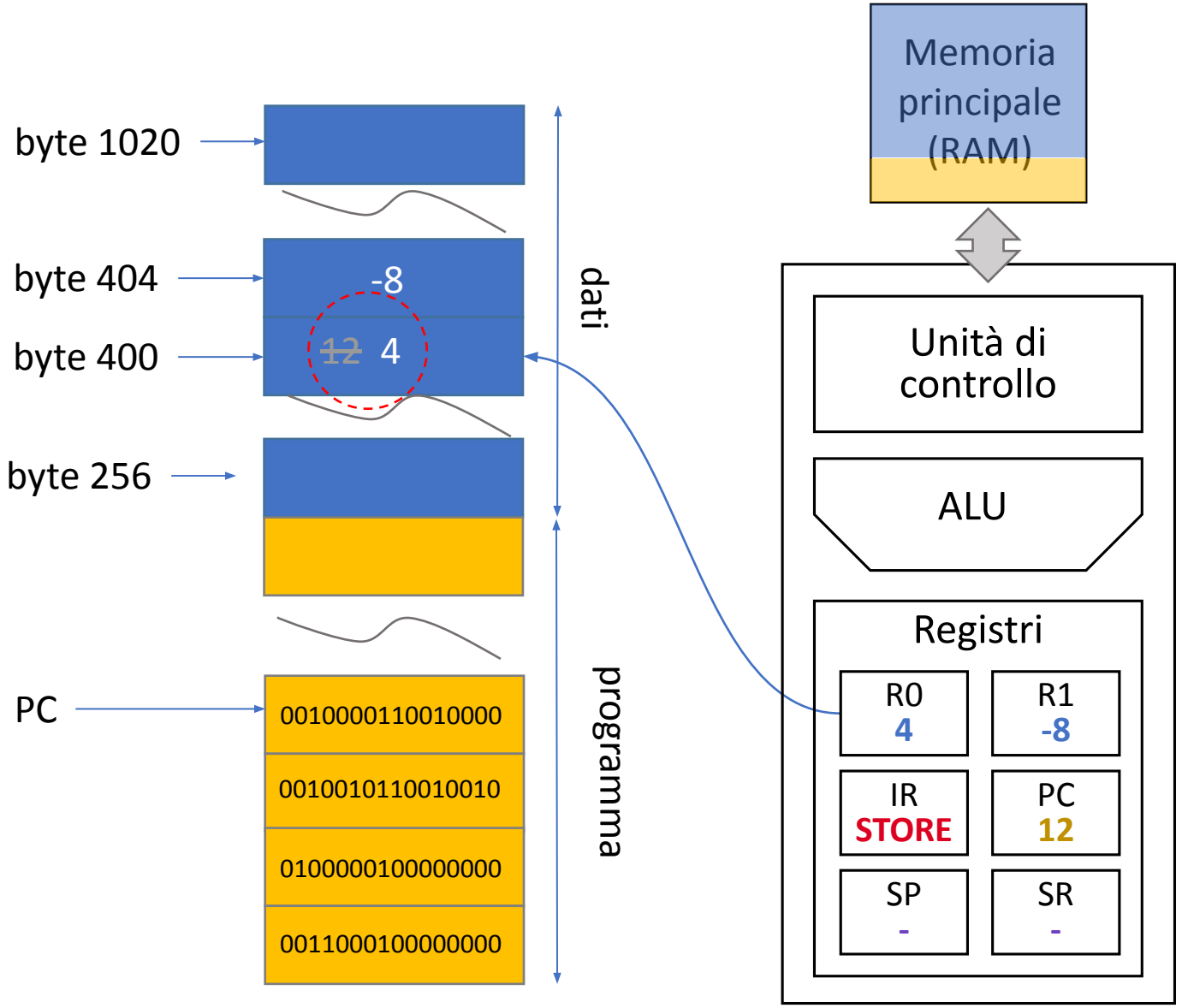
Esempio: addizione tra interi

1. LOAD, R0, @A
2. LOAD, R1, @B
3. ADD, R0, R1
4. ▶ STORE, R0, @A



Esempio: addizione tra interi

1. LOAD, R0, @A
2. LOAD, R1, @B
3. ADD, R0, R1
4. ▶ STORE, R0, @A



Esempio: moltiplicazione tra interi

Rivediamo l'esempio della moltiplicazione tramite somme ripetute, per questo linguaggio assembly.

- Moltiplichiamo due numeri interi che si trovano agli indirizzi simbolici **m** ed **n**, ed il risultato viene memorizzato all'indirizzo **m** al termine del programma
- Usiamo il linguaggio assembly per questo algoritmo di moltiplicazione, usando anche l'accumulatore **s** e il contatore delle ripetizioni **i** come visto in precedenza.

Esempio: moltiplicazione tra interi

```
1.  LOAD,   R0, 0        // inizializza R0 come accumulatore s
2.  LOAD,   R1, 0        // inizializza R1 come contatore i
3.  LOAD,   R2, @m       // carica valore indirizzo m in R2
4.  LOAD,   R3, @n       // carica valore indirizzo n in R3
5.  CMP     R1, R3       // confronta R1 ed R3, ovvero i ed n
6.  JMPEQ   <riga 10>    // Se i=n salta alla riga 10, altrimenti continua
7.  ADD,    R0, R2       // R0 ← R0 + R2, ovvero s ← s + m
8.  INC,    R1           // R1 ← R1 + 1, ovvero i ← i + 1
9.  JMP     <riga 5>     // salto incondizionato a riga 5
10. STORE,  R0, @m       // salva R0, ovvero il risultato, s all'indirizzo di m
```

Verso i linguaggi di alto livello

- Programmare in assembly è faticoso ed è facile fare errori
  - 4 righe codice per addizione, 10 righe per moltiplicazione
  - Richiede la conoscenza dell'architettura della CPU
  - Non è evidente la struttura del codice, né la logica
- Il codice scritto per una CPU *X* deve essere riscritto da capo per funzionare sulla CPU *Y* se usano linguaggi macchina differenti
- Per queste ragioni, a partire dagli anni '50 si è incominciato a sviluppare linguaggi di programmazione con istruzioni dal livello semantico più vicino al linguaggio matematico e naturale, e che permettessero di astrarre il programma dalle caratteristiche dell'hardware

Il linguaggio FORTRAN

- All'inizio degli anni '50 IBM progetta il calcolatore *modello 704* per calcoli scientifici con i requisiti seguenti:
  - Gli scienziati devono potersi focalizzare sulla programmazione di formule, ignorando i dettagli della CPU
  - I programmi devono poter essere facilmente trasportati su futuri modelli IBM senza riscrivere il programma da capo



Il linguaggio FORTRAN

- Per IBM 704 viene sviluppato il linguaggio FORmula TRANslator per superare i limiti dell'assembly
- Un compilatore traduce ogni istruzione FORTRAN in una o più istruzioni assembly specifiche per la macchina utilizzata
- Se cambia la macchina utilizzata, è sufficiente ricompilare il programma per l'hardware relativo
- Insieme a LISP, ALGOL, COBOL rappresentano i capostipiti dei «linguaggi di terza generazione», famiglia a cui appartiene il linguaggio C originario che è oggetto di questo insegnamento
- Versioni moderne di FORTRAN offrono costrutti if, while, etc. (FORTRAN 90)

Moltiplicazione tra interi in FORTRAN

```
Program Hello
INTEGER :: m
INTEGER :: n
INTEGER :: s
INTEGER :: i

WRITE(*,*) 'Inserisci m:'
READ(*,*) m
WRITE(*,*) 'Inserisci n:'
READ(*,*) n

s = 0
i = 0
do while (i<n)
    s = s + m
    i = i + 1
end do

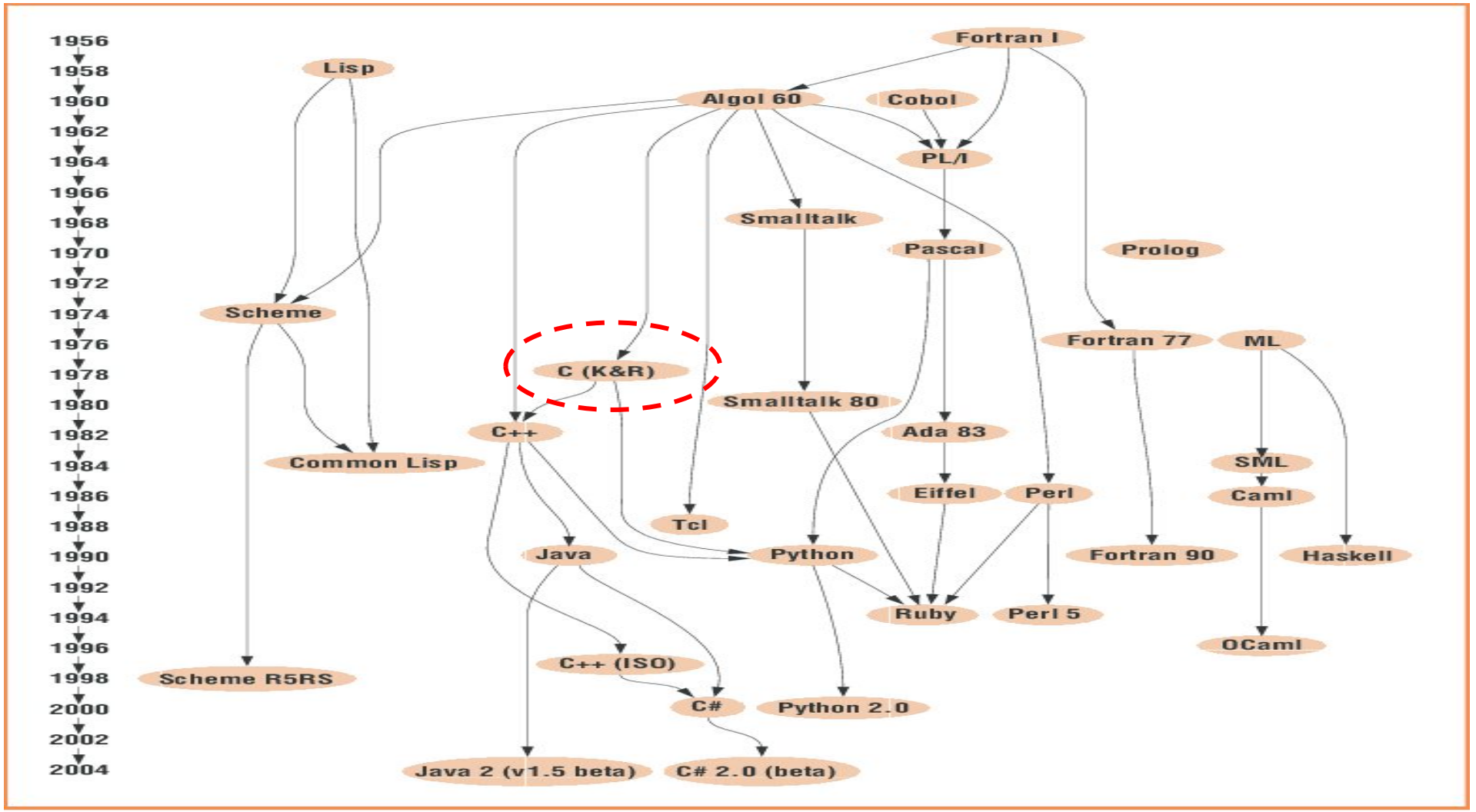
WRITE(*,*) "Risultato :",s
End Program Hello
```

1. LOAD, R0, 0
2. LOAD, R1, 0
3. LOAD, R2, m
4. LOAD, R3, n
5. CMP R1, R3
6. JMPEQ <riga 10>
7. ADD, R0, R2
8. ADD, R1, 1
9. JMP <riga 5>
10. STORE, R0, A



https://www.onlinegdb.com/online_fortran_compiler

I linguaggi di alto livello



Punti rilevanti

- Abbiamo sviluppato un semplice algoritmo per la moltiplicazione tra interi come somma ripetuta
- E' possibile programmare una macchina a basso livello (assembly) ma la scrittura di programmi è lunga e difficile
- Come vedremo, i linguaggi di alto livello come il linguaggio C nascondono molti dei dettagli specifici dell'hardware sottostante

Introduzione al linguaggio C

Storia, princìpi di base

Il linguaggio C

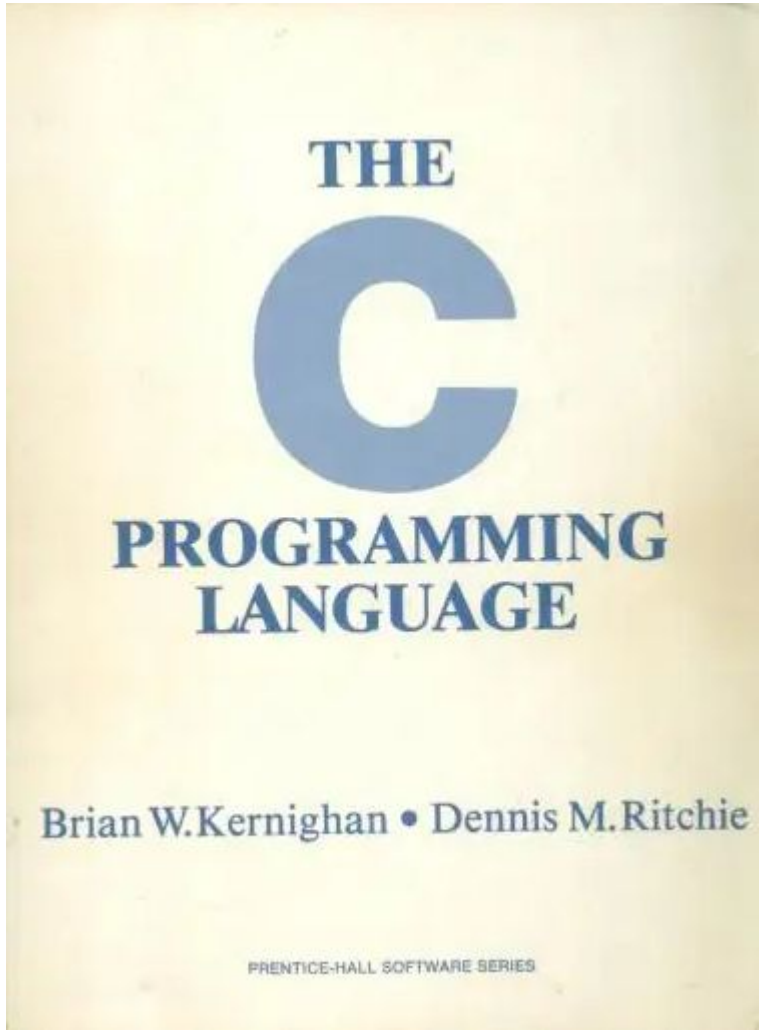
- 1969: Ken Thompson (Bell Labs, AT&T) sviluppa Unix per il minicomputer PDP-7, inizialmente scritto in linguaggio assembly.
- L'esperienza dimostra che l'assembly rende lo sviluppo di un sistema operativo oneroso e poco flessibile.
- Per superare questi limiti, Dennis Ritchie progetta il linguaggio C, concepito per unire efficienza e portabilità.
- Unix viene progressivamente riscritto in C, aprendo la strada alla sua diffusione (a partire dalle università) e influenzando in modo determinante la storia dell'informatica moderna.





Il linguaggio C

- 1972: Prima versione del C, inizialmente sviluppata per uso interno sui computer PDP-7 e PDP-11 e oggi nota come *K&R C*
- Verso la fine degli anni ‘80 è standardizzato da ANSI e ISO per uso su una varietà di hardware differente (ANSI-C, C89)
- Lo standard ha subito numerose numerosi aggiornamenti (C99, C11, C17, C23, etc.)
- Il libro di testo fa riferimento a C11



Il linguaggio C

- Nonostante la sua età, il C é un linguaggio centrale dell'informatica
- È il linguaggio di riferimento per lo sviluppo di sistemi operativi, compilatori, driver, librerie di basso livello e applicazioni ad alte prestazioni, sistemi embedded/IoT.
- Offre un controllo diretto sull'hardware e sulla memoria mantenendo al tempo stesso un livello di astrazione molto superiore all'assembly

Caratteristiche del linguaggio C

Il C è un linguaggio di programmazione:



- **compilato**: un compilatore traduce i sorgenti C nel linguaggio macchina del computer
- **imperativo**: il programma è un insieme di istruzioni pensate come ordini per il compilatore
- **strutturato**: il codice sorgente è strutturato in blocchi, racchiusi da delimitatori
- **fortemente tipizzato**: il programmatore deve specificare il *tipo* di ogni variabile

Radiografia di un semplice programma C

```c
// Un primo programma in C
#include <stdio.h>

// La funzione "main" e' il punto di ingresso del programma
int main(void) {
    printf("Buongiorno dal C.\n");
}
// fine della funzione main
```

Le linee che iniziano con '//' non sono **istruzioni** ma sono **commenti,** e sono ignorati dal compilatore

I **blocchi** strutturati del codice sono delimitati dalle parentesi graffe **{** e **}**, e contengono:

- dichiarazione di variabili
- operazioni (es: stampa in output)

Le linee che iniziano con '#' sono **direttive per il preprocessore**

Direttiva di inclusione

- Le righe che iniziano con il carattere '**#**' sono *direttive per il preprocessore* (argomento che vedremo limitatamente e sarà trattato in Programmazione II)
- La direttiva

```
#include <stdio.h>
```

  include un file che si chiama stdio.h (***st**and**d**ard **i**nput/**o**utput **h**eader*) nel nostro programma, e ne importa definizioni e contenuti.
- I files '.h' sono detti "di *intestazione*" (header) e contengono dichiarazioni di *funzioni*, ad es. fornite in librerie di sistema (printf() in libc.so)
- L'header file stdio.h dichiara funzioni come printf() e scanf()
- I nostri esempi useranno spesso vari header (talvolta li omettiamo se é ovvio, solo per ragioni di spazio nelle slide)

Radiografia di un semplice programma C



```c
1.  // Un primo programma in C
2.  #include <stdio.h>
3.
4.  // La funzione "main" e' il punto di ingresso del programma
5.  int main(void) {
6.      printf("Buongiorno dal C.\n");
7.  }
8.  // fine della funzione main
```

La **funzione main** è il punto da cui comincia l'esecuzione del programma

La funzione main riceve un input **vuoto** (void)

La funzione main ha un output di tipo intero (int). Il compilatore ci consente di ometterlo, ma potremmo scriverlo esplicitamente:

```c
return 0;
```

Funzioni

- I programmi nel linguaggio C sono organizzati in moduli detti **funzioni**, che contengono le istruzioni da eseguire
- La funzione **main** è obbligatoria ed é il punto da cui comincia l'esecuzione del programma
- Le funzioni definiscono un input ed un output
- La funzione main "ritorna" in output un numeri intero che rappresenta un codice di successo/errore al sistema operativo (non lo useremo)
- Per il momento scriveremo tutto il codice dentro il **main**

Radiografia di un semplice programma C



```
1.  // Un primo programma in C
2.  #include <stdio.h>
3.
4.  // La funzione "main" e' il punto di ingresso del programma
5.  int main(void) {
6.      printf("Buongiorno dal C.\n");
7.  }
8.  // fine della funzione main
```

Un commento che precede una funzione o delle istruzioni serve a chiarirne lo scopo.
Infatti il codice deve essere leggibile e comprensibile per un programmatore, non solo per il compilatore!

Per convenzione il codice C viene *indentato* tramite *tabulazioni* (il carattere **tab** sulla tastiera).

Le parentesi graffe delimitano il corpo (body) della funzione. Le parentesi **strutturano** l'annidamento dei blocchi e devono essere sempre bilanciate.

Programmazione strutturata

- Il codice della funzione **main**() (e di ogni altra funzione) deve essere racchiuso tra parentesi graffe **{ }**.
  Queste delimitano un blocco di istruzioni (o blocco di codice).
- Un blocco di codice è un'unità logica: può contenere
  - dichiarazioni di dati (dette variabili),
  - comandi
  - chiamate di altre funzioni
  - altri blocchi (annidamento dei blocchi), sempre delimitati dalle parentesi graffe.

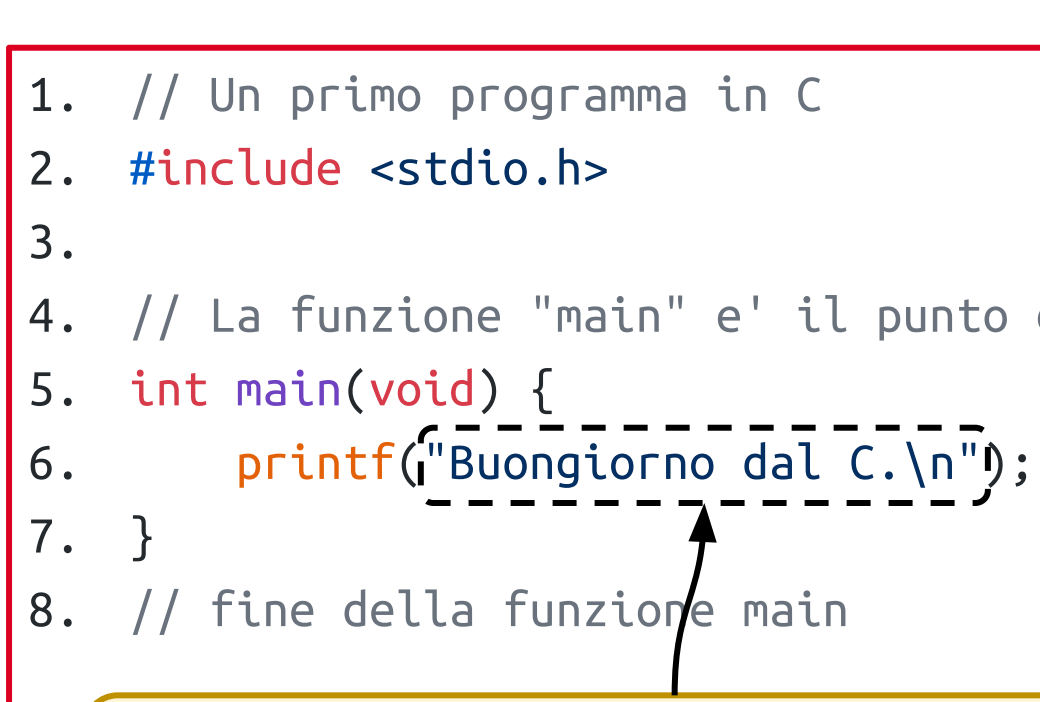
Programmare per la chiarezza

"Il codice è letto molto più spesso di quanto venga scritto: programmate per la chiarezza, non per la brevità." - Donald W. Knuth

- Per mantenere il codice chiaro è fondamentale:
  - utilizzare una corretta indentazione, che evidenzi la struttura logica (*procederemo per esempi per capire come fare e cosa significa*)
  - aggiungere commenti significativi, soprattutto per funzioni e sezioni complesse
  - scegliere nomi descrittivi per variabili e funzioni, così da rendere il codice autoesplicativo
  - evitare blocchi troppo lunghi, preferendo la scomposizione in funzioni più piccole e riutilizzabili (*vedremo più avanti nel corso come si fa*)

Imparare a scrivere codice chiaro e leggibile significa risparmiare tempo e problemi.

Radiografia di un semplice programma C



```
1.  // Un primo programma in C
2.  #include <stdio.h>
3.
4.  // La funzione "main" e' il punto di ingresso del programma
5.  int main(void) {
6.      printf("Buongiorno dal C.\n");
7.  }
8.  // fine della funzione main
```

Ogni istruzione (statement) termina con il punto e virgola “;”

Una stringa è un frammento di testo delimitato dai doppi apici (").

Una funzione (printf) viene “*chiamata*” (function call) passandogli i parametri di input.

Nelle stringhe (testo tra doppi apici "") possono comparire dei caratteri speciali dette sequenze speciali (escape sequence), che sono precedute dalla barra rovesciata ‘\’ (backslash).

La funzione printf

- Utilizzata per stampare output testuale sul terminale.
- Per ora la useremo solo con stringhe di testo racchiuse tra doppi apici " ".
- Esempio:

```
printf("Buongiorno dal C.\n");
```

- Le sequenze speciali (*escape sequence*, spesso tradotte come sequenze “di escape”) permettono di rappresentare caratteri speciali all’interno delle stringhe:
  - `\n` → nuova riga (*va a capo nella stampa a video*)
  - `\\` → stampa il carattere backslash \
  - `\t` → carattere di tabulazione (TAB)
  - `\"` → stampa il doppio apice "
  - `\0` → terminatore della stringa (lo useremo più avanti)

Dal codice al programma eseguibile

- Il linguaggio C deve essere **compilato**: il codice sorgente in formato testuale viene tradotto in linguaggio macchina, comprensibile dal computer
- Durante la compilazione, l'analizzatore lessicale (parser) suddivide il codice in token, cioè unità sintattiche (parole chiave, identificatori, operatori, simboli di punteggiatura, stringhe, costanti, ecc.)
- Come in una lingua naturale, anche i linguaggi di programmazione hanno una sintassi, più formale, che definisce le regole per costruire programmi corretti
- Se le regole sintattiche non vengono rispettate, il compilatore segnala un **errore di compilazione** e non completa la compilazione.



Sintassi del C

- Ogni istruzione deve terminare con il carattere **;**
  Errore tipico: dimenticare il ; errore di compilazione (*"expected ';' before ..."*)

```
1.  printf("Ciao")   // ERRORE: manca il ;
2.  printf("Ciao");  // CORRETTO
3.  int area = base * altezza;  // CORRETTO
```

- Le istruzioni possono occupare più righe, per leggibilità

```
1.  printf("Questo è un messaggio "
2.         "spezzato su più righe\n");
```

  È possibile andare a capo ovunque sia consentito uno spazio (tranne dentro le stringhe).
  Errore tipico: andare a capo dentro una stringa senza chiuderla → errore di compilazione (*"missing terminating " character"*).

Identificatori

- Gli identificatori servono a dare un nome agli elementi (variabili, funzioni, costanti, tipi, ...) di un programma, così da poterli riconoscere e usare.
- Negli identificatori, maiuscole e minuscole fanno differenza (case-sensitive): **Var**, **var** e **VAR** sono identificatori diversi.
- Gli identificatori devono essere scelti per *leggibilità e chiarezza*: evitare nomi troppo simili (genera confusione ed errori) o non significativi.
- Esempi di identificatori per *variabili*: `somma`, `accumulatore`

Identificatori

- Non si possono usare nomi che corrispondono alle **keywords** del linguaggio:

  auto, break, case, char, const, continue, default, do, double, else, enum, extern, float, for, goto, if, int, long, register, return, short, signed, sizeof, static, struct, switch, typedef, union, unsigned, void, volatile, while

- Non si possono usare i nomi delle funzioni della libreria standard come *main*, *printf* (anche se non viene usata, es. *sin*, *cos*) né i nomi dei simboli definiti nei file di intestazione associati alle librerie standard

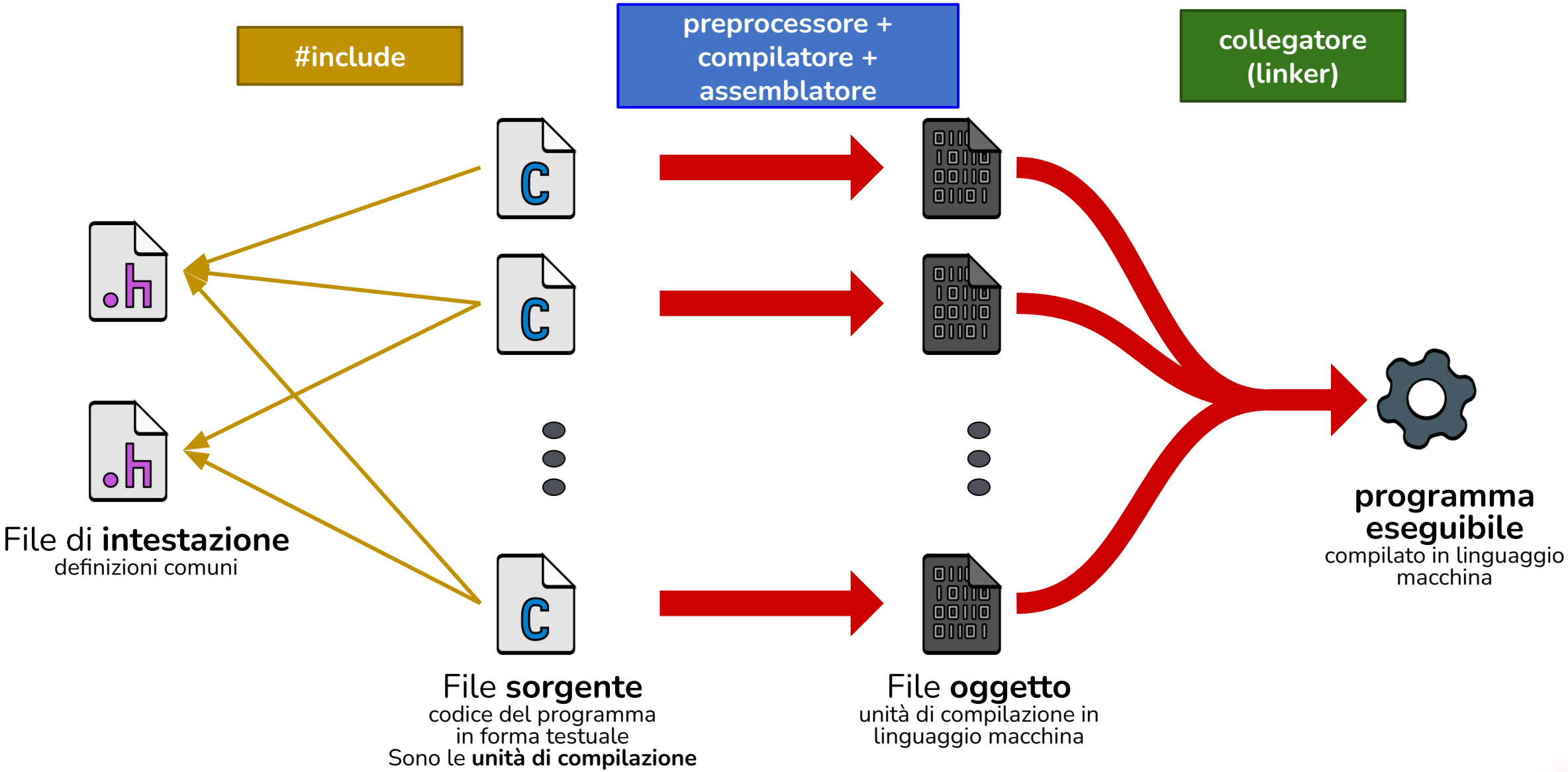
Indentazione del codice

- Le istruzioni di un blocco (NON le eventuali parentesi graffe) si scrivono tutte “indentate” (*rientrate*) di un numero fisso di spazi (es. 4), o meglio del carattere speciale *<tab>*

```
1.  if (a > 15) {
2.      x = 5;
3.      y = 2;
4.      z = a + b;
5.  }
```

- L’indentazione aiuta il programmatore a comprendere meglio il flusso del programma, per cui va fatta mentre si programma e non dopo

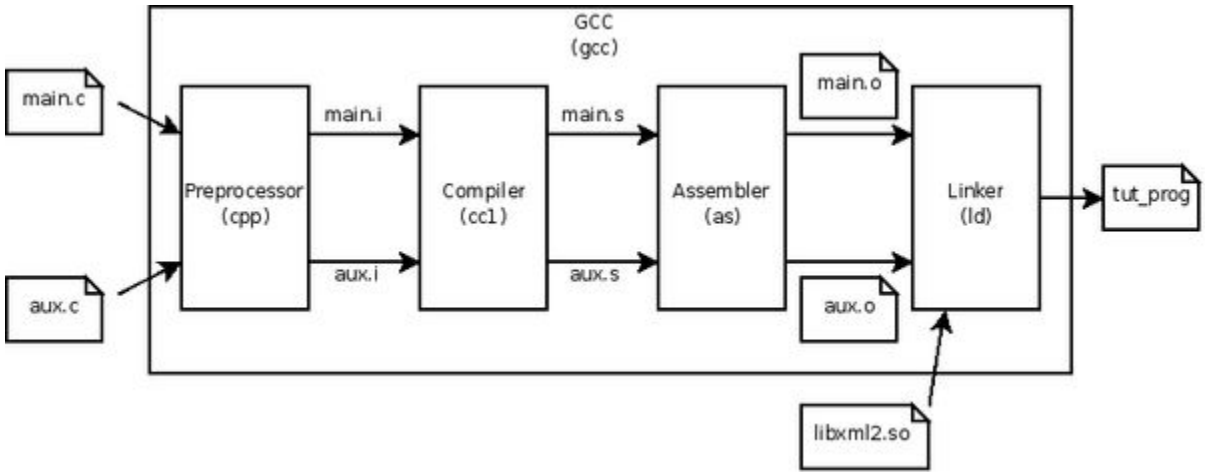
Come funziona la compilazione con il C



Il compilatore gcc



- Il compilatore che useremo è lo GNU C Compiler (GCC)
  - free, standard-compliant
  - disponibile per i maggiori sistemi operativi
  - compila per molteplici linguaggi macchina
- gcc è un frontend per un sistema di compilazione multi-stadio.



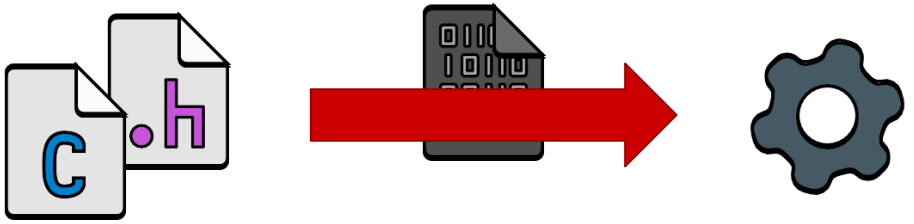
https://en.wikipedia.org/wiki/GNU_Compiler_Collection

Gli stadi della compilazione

1. Il **preprocessore** (*cpp*) elabora le direttive #include, #define, … del sorgente C producendo *sorgente intermedio*.
2. Il **compilatore** (*cc*) traduce il sorgente C in assembly, con varie opzioni
    1. ottimizzazione della velocità di esecuzione o dimensione dell'eseguibile
    2. senza ottimizzazione (per il debug, opzione *-g*)
3. L'**assemblatore** (as) produce un *file oggetto* .o in linguaggio macchina
4. Il **linker** (*ld*) unisce i moduli in linguaggio macchina in un eseguibile
    1. i file oggetto corrispondenti a sorgenti C
    2. altri file oggetto (anche compilati da sorgenti scritti in altri linguaggi)
    3. le librerie (I/O, matematiche, network, ecc.)

Compilazione ed esecuzione

Per il momento **compiliamo** con:



Unix: **gcc -Wall -Werror** *sorgente.c* **-o** *eseguibile*
Windows: **gcc -Wall -Werror** *sorgente.c* **-o** *eseguibile.exe*

Una volta che abbiamo il nostro programma compilato come eseguibile, lo possiamo **eseguire**:

Prompt dei comandi (Windows)

```
C:\Users\Elvio> buongiorno
Buongiorno!

C:\Users\Elvio>
```

Shell dei comandi (Unix)

```
elvio@linux:\home\elvio$ ./buongiorno
Buongiorno!

elvio@linux:\home\elvio$
```

Errori di compilazione ed errori a runtime

- Un programma può compilare senza **errori sintattici**, ma contenere comunque **errori** che emergono solo durante l'esecuzione (runtime).
- Gli **errori** possono avere molte cause:
  - una progettazione errata dell'algoritmo
    esempio: eseguire un ciclo prima di verificarne la condizione di terminazione
  - una realizzazione errata del programma
    esempio: divisione per zero, accesso non valido alla memoria
- È quindi importante distinguere:
  - **Errori di compilazione**: sintassi del linguaggio non rispettata, rilevati subito dal compilatore.
  - **Errori a runtime**: emergono durante l'esecuzione, dovuti a logica o implementazione scorretta.
  - **Errori logici**: il programma viene eseguito, ma non realizza correttamente il comportamento desiderato.

Errori di compilazione

Vediamo un esempio di errore di sintassi.

- Manca il punto e virgola di terminazione statement
- Il compilatore ritorna un errore di compilazione alla linea 4

```
1.  #include <stdio.h>
2.
3.  int main(void) {
4.      printf("Buongiorno dal C.\n")
5.  }
```

```
attilio@debian:~$ gcc bug_1.c -o bug1
bug_1.c: In function ‘main’:
bug_1.c:4:28: error: expected ‘;’ before ‘}’ token
   printf("Welcome to C!\n")
                            ^
                            ;
 }
 ~
```

Errori runtime

Vediamo un esempio di errore a runtime.

- Il programma compila senza errori
- Eseguire una divisione per 0 causa un errore a runtime

```
1.  #include <stdio.h>
2.  int main(void) {
3.      int x = 5;
4.      int y = 0;
5.      printf("5/2 uguale a %d\n", x/y);
6.  }
```

```
attilio@debian:~$ gcc bug_2.c -o bug2
attilio@debian:~$ ./bug2
Eccezione in virgola mobile
```

Come sviluppare il codice C

- Scrivere/modificare un sorgente con editor di testo
  esempio di editor: Notepad++
- Salvare il file sorgente in una directory con estensione .c
- Compilare il file sorgente con gcc
- Analizzare e correggere eventuali errori di sintassi
  Importante: leggete con attenzione cosa dice il compilatore
- Eseguire il file compilato
- Verificare se si comporta in modo corretto (test)
- Se ci sono problemi, modificare il codice e ripetere.

Prerequisiti per lo sviluppo in C

- È richiesta una dimestichezza di base con il PC, in particolare:
  - scrittura e gestione di file di testo
  - navigazione del filesystem
  - utilizzo essenziale della linea di comando (shell)
- Se lavorate sul vostro PC, dovrete installare un ambiente di sviluppo che includa il compilatore gcc
  (*ce ne occuperemo nelle lezioni di laboratorio*)
- In alternativa, per iniziare rapidamente senza installazioni, si può utilizzare un ambiente online:

  https://pythontutor.com/c.html#mode=edit